julesplan.md
File

repair.md
File

I want you to review this 2 files. 1 is a plan created by codex which is now partially implemented by gemini . the other is a review of these implementations and the repairs needed. this review was done by codex. can you point out where gemini is weak? I feel gemini needs some global rules to be a better coder giving codex less issues to point out in the future. also if you see any areas where codex could have provided a better plan and issues rules there. let me know.

Worked for 2m 45s

Yes. After reading both files, I think the problem is fairly clear.

The biggest Gemini weakness here is **not basic coding ability**. It is **invariant discipline**. Gemini repeatedly produces code that looks like the requested feature, compiles, and has green tests, but the tests and implementation often prove only that Gemini's own assumptions are internally consistent. Codex is repeatedly finding that the actual production contract, failure path, ownership relationship, or safety invariant was never exercised.

That is why you are seeing so many issues even though the implementation probably looks quite reasonable at first glance.

Where Gemini is weakest

1. It writes self-validating tests.

This is the most obvious recurring problem. In Phase 1, the state-machine test creates its own state list instead of exercising the production state machine. The SSE test creates its own event registry and sample payloads rather than observing actual emitted events. repair

It keeps happening. The Jules client tests reproduce the same incorrect API shapes that the implementation invented, so implementation and test agree with each other while both disagree with the real API. repair The route test named as though it verifies mounted routes doesn't make an HTTP request at all. repair Much later, the polling supervisor tests still exercise only happy-path manual ticks while missing lease expiry, stale owners, errors, shutdown, restart, etc. repair

That is probably Gemini's single biggest coding weakness in this work: **it tends to test what it wrote, rather than test the requirement against what it wrote.**

2. It optimizes for "feature-shaped code" rather than the actual acceptance criterion.

The phase drift is almost comical once you see the pattern. "Phase 6" didn't actually implement Phase 6; it implemented Phase 9-ish Jules client work while leaving the 891-line TaskManager untouched. repair "Phase 7" then did credential work instead of provider decomposition. repair The same thing continues through planned Phases 8, 9, 10, 11, 12, 13, and 14.

So Gemini seems to be treating the phase title as a general direction rather than a **contract that has to be closed**.

That then produces a downstream architectural problem: later work gets built on abstractions that were supposed to have been established earlier but never were.

3. It has a strong "plausible default" / fail-open tendency.

This shows up all over the review.

A broken vault can become `{}` rather than a typed corruption failure, and the next write can destroy the original vault. repair A Git command failure can turn into `unpushedCount: 0`. repair A failed diff operation can become an empty diff and a successful verification. repair A project with zero verification checks can become `verified: true`. repair A failed poll result is counted as a successful poll. repair

These are all manifestations of the same bad habit:

> **Unknown/error/empty is being silently converted into a convenient success-shaped value.**

That rule needs to be killed globally.

4. TypeScript types are being treated as runtime guarantees.

Gemini repeatedly casts database strings or API JSON into narrow types without actually validating them. Codex points this out in persistence, state handling, the Jules client, activities, session output, etc. repair

The Jules client literally casts `response.json()` to `T`, after which malformed responses become supposedly typed objects. repair

Gemini needs a global distinction between:

trusted internal values and **untrusted boundary values**.

Database contents, JSON responses, HTTP inputs, Git output, filenames, model responses, provider states, URLs, persisted historical data, and CLI output are all runtime data. A TypeScript assertion does nothing to validate them.

5. It loses architectural boundaries when implementing real functionality.

Codex's original plan says routes should not contain workflows, providers shouldn't write databases, generic task modules shouldn't understand Jules objects, etc. julesplan

Yet Gemini repeatedly builds new mini-monoliths. The routes contain task locking, routing, persistence, Git finalization, onboarding and process launching. repair Preflight combines Git, credentials, provider logic, application orchestration and presentation. repair The repair coordinator combines strategy, persistence, provider calls, prompt creation, state transitions and local-worker routing. repair The supervisor does the same thing again for polling. repair

This tells me Gemini understands the architecture conceptually but, when asked to deliver something functional, gravitates toward **"put everything necessary in one class/function and make it work."**

That absolutely merits a global rule.

6. It is weak around durable side effects and ambiguous outcomes.

This is especially important for Orchestra.

Gemini tends to do the remote action first and persist afterward. Session creation occurs before durable intent exists; then attempt creation, session persistence, events and task changes are separate operations. repair Repair feedback has the same problem: Jules is contacted first, and a timeout can then cause local takeover even though Jules might have accepted the request. repair

Then terminal polling persists completion before making a non-durable callback to the next workflow stage; a crash between those two can permanently strand the PR. repair

This is a distributed-systems problem, and Gemini is coding it like ordinary application code.

The global rule needs to be:

durable intent → side effect → durable acknowledgement/reconciliation, never the reverse.

7. Gemini does not naturally preserve identity/ownership invariants.

Examples include cloud sessions not being linked to execution attempts, remote session IDs not being sufficiently unique, and provider identity being hard-coded on read. repair Review functions accept independent `prNumber`, `headBranch`, and `headSha` values and can actually review a different commit than the fetched PR. repair Repair accepts independent task/session/SHA arguments and can send Task A's repair instructions to Task B's Jules session. repair

For a system like Orchestra, IDs should generally be **loaded through an owned aggregate**, not handed around independently and trusted.

8. Security work needs stronger constraints.

The credential vault is a great example. The plan explicitly says an OS-protected credential store. julesplan Gemini instead designed its own AES vault whose key can be reproduced from hostname/user/profile metadata. Codex independently regenerated the key and decrypted it. repair

Likewise, “isolated verification” ended up executing cloud-generated repository scripts directly on the Orchestra host with the entire process environment. repair

And model-to-model communication isn't consistently being treated as a trust boundary: untrusted filenames, findings, commands and outputs can flow into repair prompts. repair

So Gemini needs explicit rules saying **do not invent security mechanisms** and **everything originating outside the trusted application core is hostile data until validated**.

The global Gemini rules I would add

I would actually put something close to this into Orchestra's global Gemini coding instructions:

```
GEMINI IMPLEMENTATION RULES
```

All HTTP input, provider JSON, database values, Git output, model output
filesystem values, URLs, persisted historical data, and process output a
untrusted runtime data.
Parse and validate them before converting them into domain types.
Never use a TypeScript cast as validation.

5. FAIL CLOSED
Never convert an error, malformed value, missing response, failed comman
empty verification set, unknown state, or parse failure into success, ze
empty data, or a normal working state.
Distinguish "none", "unknown", "not configured", "failed", "unavailable"
"corrupt", and "success" explicitly.

6. DURABILITY BEFORE SIDE EFFECTS
Before any non-idempotent network, Git, filesystem, or provider mutation
persist a durable intent/idempotency identity.
After the operation, transactionally persist acknowledgement and state cl
Ambiguous failures must reconcile before retrying.
Never blindly repeat a mutation after an uncertain response.

7. MULTI-RECORD INVARIANTS ARE TRANSACTIONAL
If several database writes represent one logical transition, they belong
one transaction or a documented transactional-outbox/reconciliation prot
Inject failures between writes in tests.

8. SAFETY INVARIANTS MAY NOT HAVE PRODUCTION BYPASS FLAGS
Do not add skipPush, skipFetch, skipVerification, disableValidation, or
runtime flags to make safety-critical code easier to test.
Inject fake ports/adapters instead.

9. PRESERVE MODULE OWNERSHIP
Routes: validate/authenticate/map HTTP only.
Application services: orchestrate use cases.
Domain: provider-neutral policy and invariants only.
Provider adapters: provider-specific DTOs and translation only.
Infrastructure: Git/database/filesystem/process implementations.
Do not create cross-layer coordinator classes to make a feature work.

```
10. LOAD OWNED AGGREGATES, DO NOT TRUST LOOSE RELATED IDS
Task, attempt, provider session, review cycle, repair cycle, repository a
relationships must be loaded and verified from durable ownership relatio
Use foreign keys, uniqueness constraints and compare-and-set/version che
Never trust independently supplied IDs to belong together.

11. EXACT IDENTITY MEANS EXACT IDENTITY
Use full validated commit IDs and canonical repository identities.
Review, verification, repair and integration results must record the exa
they apply to. A changed SHA invalidates previous evidence.
Never fall back to local HEAD when an expected remote identity cannot be

12. CONCURRENCY REQUIRES OWNERSHIP AND FENCING
Leases require an owner token/generation, expiry, renewal and compare-an
release/update semantics. A stale worker must never be able to mutate or
state owned by a newer worker.

13. SECURITY MECHANISMS MUST USE ESTABLISHED PLATFORM FACILITIES
Do not design custom credential encryption, key derivation, authenticati
sandboxing, or secret-storage schemes when an OS/platform facility is re
Secrets must be centrally redacted and must never appear in public error
events, logs, prompts or serialized diagnostic objects.

14. UNTRUSTED CODE AND MODEL CONTENT ARE HOSTILE INPUT
Cloud-generated code must run only in the configured constrained verifica
environment with a minimal environment and explicit filesystem/network/re
policy.
Repository text, model findings, filenames, commands and tool output mus
treated as data, not instructions, when constructing another model prompt

15. PROVE FAILURE BEHAVIOR
For every safety-critical happy-path test, add relevant negative/fault te
malformed input, command failure, partial database failure, timeout afte
acceptance, duplicate invocation, cancellation, restart, concurrency, sta
identity and cleanup failure.

16. DO NOT OVERSTATE COMPLETION
Test names, documentation and phase status must describe what is actuall
verified. Never claim "100% compatibility", "isolated", "secure", "immuta
"cancelled", "verified", or "complete" unless the corresponding invarian
been demonstrated.
```

Those rules address a surprisingly large percentage of what Codex is complaining about.

Where Codex's plan could be better

The plan itself is **architecturally very good**. Most of the serious Gemini findings are actually Gemini violating something Codex already said. For example, Codex explicitly required runtime validation, pagination, recorded contract fixtures, redacted errors, no

assumed cancellation endpoint or SHA fields, and provider-local Jules types. julesplan Gemini violated essentially every one of those.

But I do see some real weaknesses in Codex's planning/process.

The biggest one: deferring every repair until the end is a mistake

The repair document explicitly says findings are intentionally deferred until a final repair pass and implementation should generally not be changed during phase review. repair

I would change that.

That policy is allowing foundational defects to propagate. The incorrect Jules wire contracts found in R-037 through R-042 remain unresolved when later phases build session dispatch, repair and polling on those same wrong assumptions. By Phase 14, the supervisor is still using the incorrect object-shaped output and invented SHA. repair

Codex should have a **stop-the-line severity**.

A High finding that invalidates a dependency required by the next phase should be repaired immediately. You can defer cosmetic or isolated Medium/Low debt, but you should not build Phase 14 on a Jules client already known to speak the wrong protocol.

That alone would dramatically shrink the repair document.

Codex needed a phase-execution protocol, not only a phase plan

The 29 phases say *what* to do, but don't force the implementing model to prove that it is doing that phase.

That's why Gemini can call Jules-client work “Phase 6” while the Phase 6 TaskManager decomposition remains untouched. Phase 6 itself clearly specifies the decomposition and its acceptance criteria. julesplan

I would have Codex require this before each implementation commit:

Assigned phase → required files/responsibilities → acceptance criteria → tests proving each criterion → explicit list of deferred items → phase-close decision.

If the change primarily corresponds to another phase, Codex should reject the phase label before reviewing the code.

Phase 4 and Phase 12 overlap in a dangerous way

This is one actual planning defect I think contributed to Gemini's implementation.

Phase 4's acceptance criteria already say that cloud execution metadata has a dedicated persistence model. julesplan

But Phase 12 is where Codex actually specifies the cloud execution persistence model in detail. julesplan

That creates ambiguity: should Gemini invent the cloud schema in Phase 4, or wait until Phase 12?

Gemini invented it early, and Codex subsequently found that cloud sessions weren't correctly related to attempts. repair

I'd change Phase 4 to **only extract existing persistence and establish migration/repository infrastructure**. The cloud execution schema should be introduced in Phase 12 after its identity/lifecycle model is fully defined.

And Phase 12 itself has a concrete omission: its field list includes `Orchestra task ID`, provider ID, Jules session, etc., but **does not explicitly include** `ExecutionAttempt ID`, even though the acceptance criteria immediately say multiple provider attempts may belong to a task. julesplan

Codex should have specified that foreign key explicitly.

Concurrency primitives arrive too late

Phase 13 already creates and pushes shared Git refs. Phase 16 already requires polling leases. Phase 19 creates refs/worktrees. Phase 23 merges and pushes.

Yet the full repository-level concurrency-control phase isn't until Phase 24. julesplan

That ordering is backwards.

The repository lock, managed-ref ownership, managed-worktree ownership and owner-fenced lease primitives should exist **before** features that depend on them.

I'd move the low-level concurrency/ownership infrastructure ahead of cloud dispatch—roughly before current Phase 13—and then let later phases consume it.

The same applies somewhat to cleanup hardening. Safe ownership/containment rules should exist before code starts creating temporary refs and worktrees, not after several phases already depend on cleanup.

Codex's testing requirements were too descriptive

Phase 27 has an excellent list of test layers. julesplan

But putting “comprehensive testing” at Phase 27 is psychologically dangerous for an implementation agent. Gemini has repeatedly treated earlier phase testing as “make a few focused green tests,” apparently expecting deeper testing later.

Codex should turn test quality into a **global phase gate from Phase 1 onward**, especially:

production entry point, negative test, fault injection, restart where durable state exists, concurrency where shared state exists, and adversarial/provider fixture where an external contract exists.

Similarly, Phase 2 merely says to add prohibited-import checks “where practical.” julesplan That left room for Gemini's substring-based pseudo architecture checker. Codex should have said explicitly: build the import graph, test an intentionally illegal fixture, and prove the architecture rule actually fails. Codex eventually recommends exactly that after seeing Gemini's implementation. repair

“Isolated verification” needed a stronger definition

Codex says to use a fresh worktree, deterministic dependencies, process/time/memory limits, and distinguish no-test from pass. Those are all good. julesplan

But **a worktree is Git isolation, not security isolation**.

Gemini then ran untrusted PR scripts directly on the Orchestra host with the entire environment. Codex correctly calls that out later. repair

The original plan should have said explicitly:

Untrusted cloud PR verification must not execute with Orchestra-host privileges or Orchestra's environment. Use a constrained worker/container/sandbox with an allowlisted environment, explicit network policy, filesystem boundaries and process-tree limits.

That would have removed the ambiguity.

Codex's review format should have mandated structured output

Phase 21 says Codex should return “structured findings” with severity, location, evidence, correction and blocking status. julesplan

But it doesn't explicitly say **schema-constrained JSON**, so Gemini built a Markdown parser.

That parser can produce `VERDICT: PASS` alongside a blocking finding and still report an unblocked PASS. repair

This should have been specified from the start as a versioned machine-readable schema containing at minimum `reviewedSha`, verdict, findings and invariants.

The repair-loop trust boundary also needed explicit rules

Codex's conceptual flow is right: Gemma distills findings, Jules receives them, then Orchestra waits for a new SHA and reruns verification/review. julesplan

But the plan didn't explicitly define what may enter that model-to-model prompt.

Gemini consequently formats raw review/model/repository strings into repair instructions without enough redaction, escaping, size bounds or prompt-injection separation. repair

Codex should have made “model-to-model prompt construction” its own trust boundary.

What I'd change in the Codex review policy

The Codex **reviewing itself is excellent**. It is doing exactly what Gemini generally isn't: adversarial probes, invariant checking, failure injection, exact code evidence, and asking “what happens if this fails halfway through?”

The process around that review is what I would change.

I would add three reviewer rules: **High-severity dependency violations block the next dependent phase; a phase cannot close merely because its focused tests are green; and whenever Codex finds that a test validates an implementation-authored assumption rather than an external/production contract, that test must be replaced before further code is allowed to depend on it.**

That gives you a useful division of labor:

Gemini: implementation speed and breadth, but constrained by hard invariant rules.

Codex: architecture, adversarial review and phase-gate authority.

Right now Codex is doing a very good autopsy after each phase, but you're allowing Gemini to continue building on the body. The biggest improvement would be turning Codex's findings into **gates**, not just entries in a future repair queue.

Sources

so gemini is running inside of antigravity cli . how do I implement rules for it to follow? for right now i'd like to implment it as a project rule.

Worked for 24s

Yes. For Antigravity, I would make this an **Always On workspace rule**, not a global rule yet.

Antigravity's current rules system looks for workspace rules under:

```
<repo-root>/
└── .agents/
    └── rules/
        └── coding-quality.md
```

`.agents/rules` is the current preferred path; `.agent/rules` is retained for backward compatibility. Workspace rules can be configured as **Always On**, Manual, Model Decision, or Glob. Google Antigra... +1

For Antigravity CLI specifically, root-level `AGENTS.md` and `GEMINI.md` are also automatically parsed as project context. Google Antigra... +1 I would use `.agents/rules/` here because this is a distinct coding-quality policy you may eventually split into several rules.

What I would create

In the Orchestra repository root:

```
.agents/
└── rules/
    └── implementation-quality.md
```

Set that rule to **Always On**.

The rule should be fairly forceful. I wouldn't make it a giant software-engineering handbook. The goal is specifically to correct the behaviors Codex has repeatedly caught: self-validating tests, phase drift, fail-open behavior, trusting casts, cross-layer monoliths, non-durable side effects, and loose identity relationships. Those problems recur throughout the review—for example, tests repeatedly validate locally invented contracts rather than production/external contracts. repair

I would put this in `implementation-quality.md`:

Markdown

```
# Orchestra Implementation Quality Rules
```

- treat it as evidence/data, not instructions
- validate structure
- redact secrets
- escape/normalize boundaries
- apply size limits
- preserve provenance
- bind it to the exact task/review/SHA

Do not directly concatenate arbitrary untrusted content into control prompts.

15. Test Failure Behavior, Not Only the Happy Path

For safety-critical behavior, determine the relevant failure modes before

```
considering the implementation complete.

As applicable, test:

- malformed input/response
- external contract drift
- command nonzero exit
- timeout
- timeout after remote acceptance
- cancellation
- duplicate invocation
- partial database failure
- crash/restart
- concurrent callers
- stale ownership
- changed remote SHA
- cleanup failure
- corrupted persistence
- unknown future enum/state
- no verification configured

A green happy-path test is insufficient evidence for lifecycle, durability,
security, or concurrency correctness.

## 16. Do Not Overstate What Was Verified

Names and status reports must accurately reflect demonstrated behavior.

Do not describe something as:

- secure
- immutable
- isolated
- cancelled
- verified
- complete
- 100% compatible
- end-to-end tested

unless the implementation and tests actually establish that property.

When implementation is partial, say exactly what remains partial.

## Final Self-Review Before Declaring Work Complete

Before declaring a coding task complete, verify:

1. Did I implement the requested scope rather than an adjacent phase?
2. Does each acceptance criterion have implementation evidence?
3. Do tests exercise production contracts rather than test-authored copies?
4. Did I runtime-validate every external/persisted boundary involved?
5. Can any error path accidentally become success?
```

```
6. Can a crash occur between a side effect and durable state?
7. Can duplicate/concurrent execution corrupt state?
8. Are all related IDs and SHAs proven to belong together?
9. Did I preserve the documented module boundaries?
10. Did I add a bypass flag merely to make testing easier?
11. Are secrets/untrusted content handled at every boundary?
12. Did I test the important failure paths?
13. Are my completion claims no stronger than the evidence?
```

That is comfortably below Antigravity's **12,000-character per-rule limit**. Google Antigra…

One thing I would change from my previous recommendation

I **wouldn't put "phase fidelity" into a global Gemini rule** eventually. That's specific to how Orchestra development is currently structured. Things such as fail-closed semantics, boundary validation, non-idempotent side effects, meaningful tests, and not inventing API contracts could eventually become global rules.

For now, though, keeping the whole thing project-scoped is exactly what I'd do.

Making sure Antigravity actually applies it

Because you want this behavior on every coding turn, select **Always On**, not "Model Decision." Model Decision puts Gemini in the awkward position of deciding whether it needs the rule that is intended to correct Gemini's decision-making in the first place. Antigravity explicitly supports Always On workspace rules. Google Antigra…

For the CLI, I would also add a very short root `AGENTS.md` as a belt-and-suspenders project context:

Markdown

```
# Antigravity Orchestra Project Instructions

All implementation, refactoring, debugging, testing, and repair work must foll
the mandatory workspace implementation-quality rules in:

@.agents/rules/implementation-quality.md

Before declaring a task or implementation phase complete, perform the final
self-review defined by that rule.
```

Antigravity CLI automatically consults root `AGENTS.md` / `GEMINI.md`, and Antigravity rules support `@` references to other files. Google Antigra… +1

So I'd use **both**:

```
antigravity-orchestra/
```

```
├── AGENTS.md
└── .agents/
    └── rules/
        └── implementation-quality.md
```

The actual detailed policy stays cleanly in the rule, while `AGENTS.md` makes it very difficult for a CLI session to start without being told that the policy exists.

Sources

codex-delegation.md
File

coding-principles.md
File

delegation-triggers.md
File

dev-environment.md
File

language.md
File

role-boundaries.md
File

security.md
File

testing.md
File

currently the project has these "F:\orchestra\.agents\rules\testing.md" "F:\orchestra\.agents\rules\codex-delegation.md" "F:\orchestra\.agents\rules\coding-principles.md" "F:\orchestra\.agents\rules\delegation-triggers.md" "F:\orchestra\.agents\rules\dev-environment.md" "F:\orchestra\.agents\rules\language.md" "F:\orchestra\.agents\rules\role-boundaries.md" "F:\orchestra\.agents\rules\security.md" can you tell me if any of these contradict what you are proposing?

Yes. I went through all eight, and there are a few places where your existing rules would **conflict with or weaken** the rules I proposed. Most of them are compatible, but I would modify several before adding the new implementation-quality rule.

The biggest issue is that some existing rules unintentionally encourage exactly the behavior Codex has been finding.

1. `coding-principles.md` — one important contradiction

Most of this file is fine: KISS, SRP, meaningful names, avoiding deep nesting, etc. coding-principles

The problem is this:

> “巨大な関数（50行以内を目安）” — large functions should generally be under 50 lines. coding-principles

That conflicts with the Jules architecture plan's much better rule:

> “Large files are decomposed by responsibility rather than arbitrarily split by size.” julesplan

This matters because Gemini is already prone to taking something large and “modularizing” it cosmetically rather than actually separating responsibilities. Codex found exactly that when the server monolith became several route monoliths. repair

I would replace the 50-line rule with:

```Markdown
- Do not split functions or files solely to satisfy a line-count target.
- Decompose by responsibility, dependency boundary, independently testable
  behavior, or lifecycle ownership.
- Large functions are a signal to inspect responsibility, not an automatic
  violation.
```

That is a significant change.

---

2. `testing.md` — this needs the most work

Your testing rule is not *wrong*, but parts of it encourage the exact kind of testing Codex is criticizing.

It currently emphasizes:

- Red/Green/Refactor.
- 70% unit / 20% integration / 10% E2E.
- 80–100% coverage targets.
- Mocking external dependencies. testing +2

The 70/20/10 ratio is a problem here

For Orchestra, the dangerous bugs are disproportionately at boundaries:

- HTTP routing

- SQLite transactions
- Git
- Jules API contracts
- crash/restart
- concurrency
- provider state transitions
- Codex output parsing

A rigid 70% unit target can push Gemini toward exactly what you're already getting: lots of easy unit tests and insufficient integration tests.

Codex repeatedly found tests that passed while never touching the meaningful boundary. Your Phase 5 “API test” didn't actually send an HTTP request. repair The Jules tests used hand-authored mocks reproducing incorrect API assumptions. repair

I'd remove the percentages entirely.

“Mock external dependencies” also needs qualification

It should **not** mean:

> Invent whatever Jules response is convenient and mock that.

That's exactly what went wrong.

Instead:

Markdown

```
External services may be replaced with test doubles, but the test double's
wire contract must come from authoritative documented or recorded fixtures.

Mock infrastructure, not the invariant under test.

For Git, database, HTTP, filesystem, and similar inexpensive boundaries,
prefer realistic temporary/local implementations where they materially improve
the test.

Do not mock away the boundary the test claims to verify.
```

For example, a Jules HTTP transport can absolutely be fake, but its response should be a sanitized official Jules fixture.

For Git branch safety, use a temporary bare Git repository rather than `skipPush: true`.

Coverage percentages

I'd also remove:

> New code 80%; important logic 100%; utilities 100%. testing

Coverage is useful as an indicator, but Gemini can hit 100% while testing nonsense. That's basically what your repair log demonstrates.

I'd replace it with:

```
Markdown
Coverage percentage is diagnostic, not proof of correctness.

Prioritize coverage of:
- invariants
- state transitions
- error paths
- integration boundaries
- persistence failure
- external contracts
- restart behavior
- concurrency
```

---

3. `role-boundaries.md` — slight conflict with the new rule

Your division of labor is sound:

- Antigravity/Gemini = Builder
- Codex = Designer, Debugger, Auditor. role-boundaries

But this is too absolute:

> Antigravity must not perform test design. role-boundaries

Combined with the testing rule, Gemini could interpret that as:

> “Codex didn't explicitly specify this failure test, therefore I shouldn't design one.”

That would undermine my proposed requirement that Gemini **prove failure behavior before calling implementation complete**.

I'd distinguish **test strategy** from **implementation-level regression tests**.

Something like:

```
Markdown
Codex owns test strategy for substantial features, architecture changes,
complex workflows, concurrency, durability, and security-sensitive work.
```

```
Antigravity remains responsible for implementing the test plan and MUST add
obvious implementation-level regression tests discovered while coding.

Antigravity may not weaken, replace, or redefine Codex's acceptance tests mere
to make the implementation pass.
```

That's a much cleaner division.

Codex says what must be proved.

Gemini figures out how to implement those tests and may add more when implementation exposes another edge case.

---

4. `codex-delegation.md` — generally compatible

This one fits my recommendations pretty well.

Codex handles architecture, TDD/test design, debugging, review and tradeoff analysis, while Antigravity handles file editing and straightforward implementation. codex-delegation

I would add one important principle:

Markdown

```
Codex findings that invalidate a prerequisite or safety invariant block
implementation of dependent phases until resolved.
```

Right now your actual development workflow is allowing:

> Codex: “Your Jules client speaks the wrong API.”

then:

> Gemini: “Okay, moving on to code that uses the Jules client.”

That's the structural problem I identified in the repair workflow.

I'd also add:

Markdown

```
When delegating review, provide Codex the original acceptance criteria and
assigned implementation phase, not only the changed files.
```

That would make phase drift even easier for Codex to detect.

---

5. `delegation-triggers.md` — one workflow weakness, not really a contradiction

Currently it says that after implementation Antigravity should **propose** a Codex review. delegation-triggers

For this project, I'd strengthen that.

For trivial changes, optional review is fine.

For things like:

- architecture changes
- provider integration
- persistence
- state machines
- Git operations
- concurrency
- security
- migrations
- repair/recovery
- external API contracts

Codex review should be an automatic **completion gate**, not merely something Gemini offers.

I'd change Check 4 roughly to:

```
Implementation complete?
    ↓
Does it affect architecture, persistence, external APIs, Git, concurrenc
security, lifecycle/state, recovery, or >1 module?
    → Yes: Codex review is required before declaring complete.
    → No: Codex review may be suggested.
```

That would be valuable.

---

6. `security.md` — strongly aligned, but too shallow for Orchestra

There's no meaningful contradiction with my proposed rule. Your existing rule already says external input must be validated. security

It also correctly keeps Codex read-only. security

But it needs two additions because of what Codex found.

First:

```
Markdown

Do not design custom cryptographic secret-storage mechanisms when the
architecture calls for OS-backed/platform credential storage.
```

That would have prevented Gemini's machine-metadata-derived AES vault.

Second:

```
Markdown

Cloud-generated or otherwise untrusted repository code must not execute with
Orchestra's full host credentials/environment merely because it is running in
a separate Git worktree.
```

A worktree is not a sandbox. Codex found Gemini doing exactly that. repair

I'd also expand “external input” to explicitly include:

- database values
- Git output
- provider responses
- model output
- filenames/paths
- persisted historical values

because Gemini seems to interpret “input validation” primarily as user HTTP input.

---

7. `dev-environment.md` — there is a practical conflict

This says:

> use `pnpm` for JavaScript/TypeScript. dev-environment

But the Jules plan explicitly talks about npm projects using `npm ci` when a compatible lockfile exists. julesplan

More importantly, package manager choice should follow **the repository's existing lockfile**, not a blanket preference.

I would change that rule to:

Markdown

```
Use the package manager already selected by the repository and its lockfile.

- package-lock.json -> npm
- pnpm-lock.yaml -> pnpm
- yarn.lock -> Yarn

Do not introduce or replace a package manager without an explicit migration
decision.
```

That's much safer.

Same with Python: don't automatically introduce `uv` into an established pip/Poetry project just because the global project rule says uv.

---

8. `language.md` — no coding conflict

Nothing there conflicts materially with what I'm proposing. It governs communication language and Codex prompts. language

I wouldn't touch it for this effort.

So I would not just add my new rule unchanged

I'd make the rule set complementary rather than duplicate everything.

Your current structure is actually good:

```
coding-principles.md
testing.md
security.md
role-boundaries.md
codex-delegation.md
delegation-triggers.md
dev-environment.md
language.md
```

Then add:

```
implementation-integrity.md
```

And let that new file focus specifically on the missing behaviors:

production contract > implementation assumptions
fail closed
runtime boundary validation
durable-before-side-effect
transactional invariants
ownership/identity validation
exact SHA binding
fenced concurrency
no safety bypass flags
phase fidelity
no cross-layer coordinator monoliths
no overstating completion

I would *not* duplicate generic SRP, TDD, security basics, naming, or Codex role definitions there.

The four existing files I would modify

If this were my repo, I'd change:

1. `coding-principles.md` — remove the 50-line pseudo-limit and explicitly favor responsibility-based decomposition.
2. `testing.md` — remove 70/20/10 and hard coverage goals; add production-contract, boundary, failure-injection and authoritative-fixture rules.
3. `role-boundaries.md` — clarify that Codex owns test strategy but Gemini still owns implementation/regression testing.
4. `dev-environment.md` — follow repository lockfiles instead of mandating pnpm/uv.

Then I'd make a smaller addition to `security.md` and strengthen `delegation-triggers.md` so significant implementation work requires Codex review rather than merely suggesting it.

That combination would be considerably better than simply stacking another large rule file on top of the current ones.

Sources